Time-Off Microservice — Loopholes & Known Gaps

ExampleHR Engineering

2026-05-12

About This Document

This is an honest assessment of every gap, edge case, and potential failure mode in the current implementation. Issues are rated by **Severity** (impact if it occurs) and **Likelihood** (probability under normal usage).

Critical Gaps

1. HCM Deduct Succeeds, DB Write Fails (Phantom Deduction)

Severity: HIGH | Likelihood: VERY LOW

What happens: During approval, `HcmClientService.deductBalance` succeeds and the HCM deducts the days from Alice's balance. Immediately after, all DB retries (up to 5) fail due to SQLite errors. The service returns a `500` to the manager.

Effect: The HCM has deducted the days, but ExampleHR's request is still `PENDING`. Alice's visible balance in ExampleHR is now incorrect — it shows more days than she actually has. The manager tries to approve again; the HCM deducts again; Alice is now double-deducted.

Current mitigation: The code logs a `CRITICAL` error with the HCM reference ID. A human must manually call `POST /api/v1/hcm/sync/single` or wait for the next batch sync to reconcile.

Proper fix: Outbox pattern — write the HCM submission intent to a `hcm_pending_submissions` table in the same DB transaction. A background worker reads it and calls HCM, marking it complete on success. This decouples the HCM call from the user-facing request.

---

2. No Real Authentication or Authorization

Severity: HIGH | Likelihood: CERTAIN if deployed as-is

What happens: The `JwtAuthGuard` checks only that an `Authorization: Bearer <anything>` header is present. Any string passes.

Consequences: - Any authenticated employee can approve or reject any other employee's requests. - Any authenticated user can read any employee's balance. - There is no manager-vs-employee role separation.

Current mitigation: None — this is an acknowledged scope cut.

Proper fix: Verify the JWT (using `@nestjs/jwt`), extract `sub` (employee ID) and `role` from claims, and add authorization checks: employees can only read/create/cancel their own requests; managers can approve/reject requests for their direct reports.

Significant Gaps

3. No Idempotency on Approve / Reject / Cancel

Severity: MEDIUM | Likelihood: LOW

What happens: If a manager clicks "Approve" and the response is lost (network timeout), they click again. The second approval call: 1. Loads the request (already `APPROVED`) 2. Finds it's not `PENDING` 3. Returns a `409 Conflict`

The manager sees an error even though the first approval succeeded. This is technically correct but creates a confusing UX — the manager doesn't know if the first attempt worked.

The deeper issue: If the HCM deduct succeeded on the first call but the DB write failed (see gap #1), the manager retrying will cause a second HCM deduct.

Proper fix: Accept an `Idempotency-Key` header on `PATCH /:id/approve` (and reject/cancel). Cache the first response by key+requestId for 24 hours.

---

4. Batch Sync Discrepancy: No Resolution Path

Severity: MEDIUM | Likelihood: LOW

What happens: A batch sync arrives where HCM shows Alice has 3 days but ExampleHR has 5 pending days. The system logs a `WARN` and continues. No alert is sent, no admin dashboard shows it, no auto-resolution runs.

Effect: Alice's `available_days` computes to `-2`. If she tries to request more leave, she's correctly rejected. But no one is proactively notified, and the discrepancy could persist indefinitely.

Proper fix: 1. Write discrepancies to a `balance_discrepancies` table for admin review. 2. Add an admin endpoint `GET /api/v1/admin/discrepancies` to surface open items. 3. Optionally auto-trigger a single-balance real-time refresh after a discrepancy is detected.

---

5. Partial Batch Sync Failure

Severity: MEDIUM | Likelihood: LOW

What happens: A batch sync payload has 500 entries. Entry #347 causes an unexpected error (e.g., a DB constraint not caught by the application). The error is thrown, the batch sync returns a `500`, and the HCM considers the batch failed. But entries 1–346 have *already been committed* to the DB.

Effect: The HCM might retry the entire batch. Entries 1–346 will be processed again (upsert, so harmless) but depending on timing, the retry might use a stale `syncedAt` timestamp, causing `hcm_last_synced_at` to go backwards.

Current mitigation: Each entry is in its own transaction, so only the failing entry is rolled back, not the whole batch. The response does include `{ processed, discrepanciesFound }` but doesn't distinguish partial failure from complete success.

Proper fix: Track which entries failed and return a partial-success response (`207 Multi-Status`) with per-entry success/failure. The HCM can then retry only the failed entries.

---

6. Leave Type Is Not Validated Against HCM

Severity: LOW | Likelihood: MEDIUM

What happens: A request is submitted with `leaveType: "SABBATICAL"`. ExampleHR accepts it, creates a `leave_balance` row for it, and reserves the days. When approval is attempted, the HCM returns a `422` with "invalid leave type" — which ExampleHR correctly turns into a `409`. But the balance row for `"SABBATICAL"` now exists with `pendingDays > 0` and will never be approved.

Effect: The employee's balance looks slightly wrong until they cancel the request. Not catastrophic, but confusing.

Proper fix: Validate leave types against an allowed list fetched from the HCM at startup (or periodically cached). Reject invalid leave types at request creation time with a `422`.

---

7. No Cleanup of Long-Lived PENDING Requests

Severity: LOW | Likelihood: HIGH over time

What happens: A manager never acts on a request. The request stays `PENDING` indefinitely. The days are reserved in `pending_days` forever. Alice's balance is permanently reduced by those reserved days.

Effect: Over months, employees accumulate "ghost reservations" from never-actioned requests. Their visible balance is artificially lower than their actual entitlement.

Proper fix: A scheduled job (`@nestjs/schedule`) that auto-expires `PENDING` requests older than N days (configurable per company policy) by transitioning them to `CANCELLED` and releasing the pending reservation.

---

8. `hcm_balance` Can Drift Without Periodic Reconciliation

Severity: LOW | Likelihood: MEDIUM

What happens: The HCM changes Alice's balance (holiday carryover, manual correction) and doesn't push a sync. If ExampleHR's batch sync schedule is every 24 hours, Alice could see an incorrect balance for up to a day.

Current mitigation: Real-time single-balance webhook exists. Employees can trigger a manual refresh via `POST /employees/:id/balances/refresh`. But neither is guaranteed.

Proper fix: A scheduled job that calls `HcmClientService.getBalance` for all employees who haven't been synced in the last N hours, updating the shadow. Configurable polling frequency.

---

9. The `days` Field Is Caller-Supplied, Not Computed

Severity: LOW | Likelihood: MEDIUM

What happens: A client submits `startDate: "2026-06-01"`, `endDate: "2026-06-03"` but `days: 10`. ExampleHR accepts it, reserving 10 days for what should be a 2-day request.

Current mitigation: None — the validation only checks `days >= 0.5`.

Proper fix: Compute `days` from `startDate` and `endDate` on the server side (considering business days and company calendar), or add a server-side validation that checks `days <= (endDate - startDate + 1) × some_factor`. The proper fix requires integrating a company calendar, which is HCM-specific.

Minor Gaps

10. No Soft Delete or Request History

Requests in terminal states (`REJECTED`, `CANCELLED`, `COMPLETED`) stay in the `time_off_requests` table forever. There is no archival mechanism. For a company with thousands of employees over years, this table will grow unbounded.

11. SQLite File Growth

SQLite WAL mode can leave behind `-wal` and `-shm` files if the process crashes. There's no automatic WAL checkpoint configuration. For long-running production deployments, `PRAGMA wal_autocheckpoint` should be configured.

12. No API Versioning Strategy Beyond `/v1`

The current version prefix is `/api/v1/`. There is no deprecation policy, no version sunset mechanism, and no schema evolution strategy for when v2 is needed.

Summary Table

#	Issue	Severity	Likelihood	Fix Complexity
1	Phantom deduction (HCM deduct + DB fail)	HIGH	Very Low	High (outbox)
2	No real auth/RBAC	HIGH	Certain	Medium
3	No idempotency on approve/reject/cancel	MEDIUM	Low	Low
4	Discrepancy has no resolution path	MEDIUM	Low	Medium
5	Partial batch sync failure reporting	MEDIUM	Low	Medium
6	Leave type not validated against HCM	LOW	Medium	Low
7	PENDING requests never expire	LOW	High	Low
8	hcm_balance drifts without polling	LOW	Medium	Low
9	days field caller-supplied, not validated	LOW	Medium	Medium
10	No request archival	LOW	High	Low
11	SQLite WAL not checkpointed	LOW	Low	Trivial
12	No API versioning strategy	LOW	Low	Low

Critical priority for production: Issues #1 and #2 must be addressed before production deployment. All others are improvements.